

Concave Hull Revisited: Runtime Analysis and Acceleration of the K-Nearest Neighbours Approach

Sean Perman¹ and Mario Lopez¹

¹*Ritchie School of Engineering and Computer Science, University of Denver, Denver, Colorado, U.S.A.
sean.perman@du.edu*

Keywords: Concave Hull, Convex Hull, Polygon, Contour, K-Nearest Neighbours, Spatial Bucketing, Computational Geometry.

Abstract: Moreira and Santos introduced a k -nearest-neighbours variation of gift wrapping for constructing concave hulls. The parameter k heuristically controls concavity. When construction fails, the algorithm discards the partial hull and restarts with a larger k until it finds a valid hull. The original paper lacked an asymptotic runtime analysis. We show that one fixed- k attempt has a worst-case upper bound of $O(kn^2)$, linear k -growth gives $O(n^4)$, and constant-ratio geometric growth reduces the restart bound to $O(n^3)$ under the same cost model. We make termination explicit by testing $k = n - 1$ once and then using a standard convex-hull fallback if every concave-hull attempt fails. The analysis identifies edge-intersection checking and repeated reconstruction as main costs. In a two-factor experiment on real and synthetic datasets, uniform-grid spatial bucketing gives paired median speedups of $1.40\text{--}3.41\times$ while preserving the ordered hull, whereas geometric growth gives $1.36\text{--}44.57\times$ speedups but usually selects a different valid hull. Combining both gives $1.89\text{--}147.94\times$ speedups over the baseline. Finally, counterexamples show that neither hull concavity nor fixed- k validity is monotone in k , ruling out ordinary binary search for the smallest valid k . These results provide runtime bounds, acceleration strategies, and clarify how k affects construction.

1 INTRODUCTION

Given a finite set of points in the plane, many applications need a polygon that represents the region covered by those points. The convex hull is the unique smallest convex polygon containing the entire point set, and it is useful when the main goal is enclosure. However, it may not represent the shape of the data well when the points contain indentations, gaps, or separate groups. A concave hull relaxes the requirement of convexity, allowing the boundary to bend inward and follow the data more closely. Unlike the convex hull, a concave hull is not unique: many different polygons can represent the same point set with different levels of detail. Concave hull algorithms therefore use parameters or geometric rules to control the resulting boundary (Galton and Duckham, 2006; Duckham et al., 2008).

Moreira and Santos proposed a concave hull algorithm based on k -nearest neighbours and gift wrapping (Moreira and Santos, 2007). When the algorithm fails to construct a valid hull, it discards its partial result and starts again with a larger value of k . This process continues until it finds a hull containing ev-

ery input point. Although the algorithm is simple and flexible, the original paper did not provide an asymptotic runtime analysis. Repeated restarts and edge-intersection checks can cause the algorithm to perform a large amount of repeated work.

This paper makes three contributions. First, we show that one attempt with fixed k has a worst-case upper bound of $O(kn^2)$, and that repeated attempts under linear k -growth give an $O(n^4)$ upper bound. A constant-ratio geometric schedule instead gives an $O(n^3)$ upper bound under the same model. We also identify a termination gap in the published recursive pseudocode and close it by testing $k = n - 1$ once, then returning the ordinary convex hull if the concave construction still fails. Second, we evaluate the two independent optimization targets exposed by the analysis. Uniform-grid spatial bucketing accelerates edge-intersection checking by $1.40\text{--}3.41\times$ without changing the hull. Geometric growth reduces the number of reconstruction attempts and gives $1.36\text{--}44.57\times$ speedups, while sometimes changing the selected valid hull; their combination is $1.89\text{--}147.94\times$ faster than the original baseline. Third, we give counterexamples showing that increasing k can produce a

more concave hull and that success at one value of k does not imply success at every larger value.

2 THE MOREIRA-SANTOS ALGORITHM

2.1 Main Idea

Moreira and Santos (Moreira and Santos, 2007) proposed a simple and intuitive concave hull algorithm based on the idea of modifying Jarvis march (Jarvis, 1973). Instead of selecting the next hull vertex from all points, their method restricts attention to the k -nearest neighbours of the current vertex. Among these **candidates**, it chooses the one that creates the largest right-hand turn without introducing an intersection with the existing hull. This restriction makes it possible to construct non-convex boundaries while preserving the local, step-by-step flavor of gift wrapping (Jarvis, 1973). The parameter k governs how local the search is: smaller values tend to produce tighter and more detailed hulls, while larger values yield smoother and more convex boundaries.

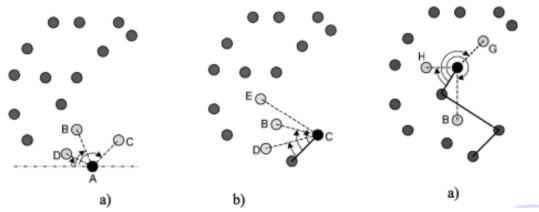


Figure 1: The k -nearest neighbours approach.

The algorithm begins by selecting an extreme point (minimum y -coordinate) as the starting vertex. From the current hull vertex, it computes the k -nearest neighbours that have not been excluded from consideration. These candidates are then ordered according to the turning angle relative to the previous hull edge. A greedy approach is then used to select a **valid candidate** that produces the largest right-hand turn without intersecting the existing hull. This process repeats until the walk returns to the starting point, thereby closing the polygon.

2.2 Failure Cases and Restart Logic

The algorithm has two main failure cases. First, there may be no valid candidates among the current k -nearest neighbours: each candidate edge intersects the existing partial hull, so the algorithm cannot safely

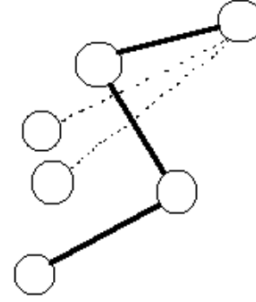


Figure 2: All candidate edges intersect the current hull.

extend the boundary (**dead end**). Second, after the hull has been closed, some input points may still remain outside the polygon. In that case, the computed hull fails to enclose the full point set (**excluded points**). The published algorithm responds to either failure by restarting with $k + 1$. Our implementation preserves its rule that the starting point cannot be reintroduced before the fourth vertex. It changes only the terminal behavior: $k = n - 1$ is attempted once, and a failed attempt at that cap returns the ordinary convex hull rather than recursively requesting the same capped value again.

2.3 The Role of the Parameter k

The parameter k is the central control variable in the algorithm. When k is small, candidate selection is highly local, allowing the boundary to follow fine geometric detail and concave features. When k is larger, the set of candidate vertices broadens, and the resulting hull tends to be smoother and closer to a convex enclosure. For n distinct input points, $k = n - 1$ is the largest meaningful value because every available point can then be considered. This makes the vertex choice resemble Jarvis march, but does not by itself prove that the complete procedure terminates under the original closure rule. An important feature of the

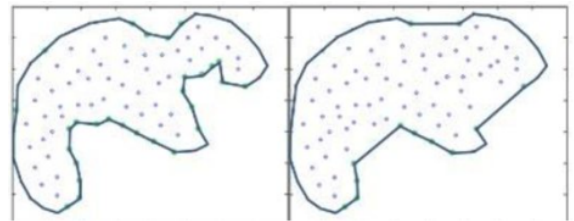


Figure 3: (left = low k), (right = high k).

method is that k is dimensionless: it is not a geometric distance threshold, but rather a count of neighbours. This distinguishes it from approaches such as alpha

shapes, where the main parameter is directly tied to spatial scale and may be sensitive to sampling density (Edelsbrunner et al., 1983). In the Moreira–Santos method, the parameter has a more combinatorial interpretation, which can make it easier to reason about across differently scaled datasets.

2.4 Comparison with Other Concave Hull Approaches

Compared with alpha-shape-based methods and related shape reconstruction techniques, the Moreira–Santos algorithm has the advantage of conceptual simplicity and a parameter that is not directly tied to metric scale. In contrast, methods based on geometric thresholds often require careful tuning relative to point spacing or sampling density (Edelsbrunner et al., 1983; Liao et al., 2021). The k -nearest-neighbours approach is therefore appealing both practically and theoretically. It offers a direct geometric construction, adapts automatically by increasing k when necessary, and provides a natural bridge between highly detailed concave boundaries and more convex approximations. These properties make it a useful baseline for the improvements developed in the later sections of this paper.

3 RUNTIME ANALYSIS OF THE ORIGINAL ALGORITHM

3.1 Cost of One Successful or Failed Run with Fixed k

We first analyze one execution of the original Moreira–Santos algorithm with a fixed neighbour parameter k . Let n denote the number of input points, and let s denote the number of steps completed before that run either closes a hull or fails and triggers a restart. Since each non-closing step adds a new hull vertex and removes a point from further consideration, we have $s = O(n)$. At each step, the algorithm performs three main operations:

Nearest-neighbour search. At each step, the algorithm computes the k -nearest neighbours of the current point. The original paper’s Mathematica package invokes a helper named `NearestPoints`; it specifies Euclidean distance, but neither identifies this helper as a Wolfram built-in nor reports its data structure or selection algorithm (Moreira and Santos, 2007). To give the implementation the benefit of the doubt, we

model exact neighbour selection as linear and charge no separate index-construction or rebuilding cost:

$$T_{\text{knn}}(i) = O(n).$$

This is also a valid worst-case bound for exact tree-based search. Granting the more optimistic expected indexed-query cost $O(\log n + k)$ would only reduce the nearest-neighbour contribution; the intersection term derived below would still dominate.

Sorting the candidates. Once the k nearest neighbours have been selected, they are sorted by angle relative to the previous hull direction. Sorting k items costs

$$T_{\text{sort}}(i) = O(k \log k).$$

Intersection checking. Suppose the partial hull currently contains i vertices. A candidate edge must be checked against $O(i)$ existing hull edges. Since the algorithm may test as many as k candidates before finding a valid one or concluding that no valid candidate exists, the intersection-checking cost at step i is

$$T_{\text{check}}(i) = O(ki).$$

Therefore, the total cost at step i is

$$T(i) = O(n) + O(k \log k) + O(ki).$$

Summing over all s steps in the run gives

$$C_{\text{run}}(k) = \sum_{i=1}^s \left(O(n) + O(k \log k) + O(ki) \right).$$

Separating the three contributions:

$$C_{\text{run}}(k) = O(sn) + O(sk \log k) + O\left(k \sum_{i=1}^s i\right).$$

Using the triangular-sum identity, $\sum_{i=1}^s i = O(s^2)$, we obtain

$$C_{\text{run}}(k) = O(sn) + O(sk \log k) + O(ks^2).$$

Since $s = O(n)$, this yields

$$C_{\text{run}}(k) = O(n^2) + O(nk \log k) + O(kn^2).$$

A completed run also performs a final containment check to verify that all points lie inside the constructed polygon. In the worst case, this adds at most $O(n^2)$, which does not change the asymptotic bound above. Since $k \geq 1$, the term $O(kn^2)$ dominates, giving a single run with fixed k worst-case cost:

$$C_{\text{run}}(k) = O(kn^2).$$

3.2 Cost of Restarts

The original algorithm may restart with a larger value of k in two cases: first, when all current candidates would produce an intersecting edge (**dead end**), and second, when a closed hull is found but at least one input point lies outside the resulting polygon (**excluded points**). In either case, the algorithm discards the current hull and restarts from the beginning with $k + 1$. Suppose the algorithm begins with k_0 and eventually succeeds only after trying the sequence $k_0, k_0 + 1, k_0 + 2, \dots, K$. Then the total cost is:

$$C_{\text{total}} = \sum_{k=k_0}^K C_{\text{run}}(k) = O\left(n^2 \sum_{k=k_0}^K k\right).$$

In the worst case, the algorithm restarts may increase k until $K = n - 1$. Therefore,

$$C_{\text{total}} = O\left(n^2 \sum_{k=1}^{n-1} k\right) = O(n^2 \cdot n^2) = \boxed{O(n^4)}.$$

This bound depends on the original linear update $k \leftarrow k + 1$. Consider instead a constant-ratio geometric schedule

$$k_{j+1} = \min\{\lceil rk_j \rceil, n - 1\}, \quad r > 1.$$

The tested values form a geometric series, so $\sum_j k_j = O(n)$. Applying the same per-run bound gives

$$C_{\text{geo}} = O\left(n^2 \sum_j k_j\right) = \boxed{O(n^3)}.$$

This is an upper bound for a modified growth schedule, not for the original algorithm. The schedule may skip smaller successful values of k and therefore change the returned hull. The schedule is finite because the all-candidates value is tested once; if that attempt also fails, the convex-hull fallback described below is used. Its $O(n \log n)$ cost does not change either restart bound. These bounds identify two independent optimization targets. Within an attempt, edge-intersection checking drives the $O(kn^2)$ term; across attempts, the growth schedule determines the restart sum. Section 4 addresses the first target with spatial bucketing. Geometric growth addresses the second, but may skip smaller successful values of k ; Section 6 shows that ordinary binary search cannot recover the minimum in general because fixed- k validity is not monotone.

3.3 Termination

The original paper states that repeatedly increasing k eventually produces a valid hull, but its pseudocode

does not establish this for every input. The value of k is capped at $n - 1$, yet a failed attempt at that value recursively calls the algorithm with $k + 1$, which is immediately reduced back to $n - 1$. The same failed attempt may therefore be repeated indefinitely. Boundedness of k alone therefore does not imply termination.

Our restart-based implementation uses the following terminal rule. The construction attempts each value in its growth schedule at most once, including a final attempt at $k = n - 1$. If no attempt returns a valid enclosing polygon, it computes the ordinary convex hull with Andrew's monotone-chain algorithm (Andrew, 1979).

Proposition. For every finite input point set, the modified restart-based algorithm terminates. If the set is non-collinear, it returns a simple polygon containing every input point; if the set is collinear, it terminates with an explicit report that no two-dimensional polygonal hull exists.

Proof. Remove duplicate points and let n be the number that remain. During a fixed- k attempt, each accepted non-starting vertex is removed from the candidate set. Consequently, after finitely many iterations the attempt either closes its polygon, reaches a dead end, or exhausts its candidates. Under either restart schedule analyzed here, k strictly increases and never exceeds $n - 1$, which is attempted only once, so the schedule contains finitely many attempts. If one of them produces a valid enclosing polygon, the algorithm stops. Otherwise, the convex-hull fallback terminates. For a non-collinear finite set its result has at least three vertices, is simple, and contains every input point. For a collinear set the convex hull is a segment, which the algorithm detects and reports instead of claiming a polygon. Thus every execution terminates, and every successful execution returns an enclosing polygon. $\square$

4 SPATIAL BUCKETING IMPROVEMENT

The runtime analysis identifies edge-intersection checking as one of the main costs within each construction attempt. When the partial hull contains h edges, a naive implementation may compare every candidate edge against all h existing edges. Many of these comparisons are unnecessary because an edge located far from the candidate cannot intersect it. We reduce this work using uniform-grid spatial bucket-

ing, a standard technique for limiting geometric tests to nearby objects (Franklin et al., 1989).

The method divides the plane into square grid cells. When an edge is added to the hull, a reference to it is stored in every cell containing any part of that edge. When testing a new candidate edge, the algorithm looks at the cells containing the candidate and applies the exact intersection test only to the existing hull edges referenced in those cells. An existing edge referenced in multiple cells is checked only once. Because intersecting edges share at least one cell, this filtering does not miss any intersections. Points on grid boundaries are associated with the adjacent cells so that boundary intersections are also detected.

For the experiments, the grid-cell width is selected automatically as

$$w = \frac{\max\{\Delta_x, \Delta_y\}}{\sqrt{n}},$$

where Δ_x and Δ_y are the coordinate ranges of the input and n is the number of input points. This choice scales the grid according to both the spatial extent of the data and the number of points.

Let g be the number of grid cells containing part of a candidate edge, and let i be the number of distinct existing hull edges retrieved from those cells. Assuming constant-time grid lookups, processing one candidate requires $O(g + i)$ work, including the exact tests against the retrieved edges. In comparison, the naive implementation may require $O(h)$ exact tests. The worst-case bound is unchanged because i can still be as large as h . For spatially distributed hull edges, however, i is usually much smaller than h . Spatial bucketing is therefore a practical reduction in intersection work rather than an improvement to the worst-case complexity class. Section 5 evaluates its effect on intersection counts and running time.

5 EXPERIMENTAL EVALUATION

We implemented a configurable version of the original algorithm and evaluated two changes: spatial bucketing and geometric growth of k . Spatial bucketing replaces the naive scan of all hull edges, while geometric growth replaces the original update rule $k \leftarrow k + 1$ with

$$k \leftarrow \min\{\lceil 2k \rceil, n - 1\}.$$

Combining these choices produces four configurations: the original algorithm, bucketing only, geometric growth only, and both improvements together.

To ensure a fair comparison, all four configurations begin with $k_0 = 3$, use the same SciPy `cKDTree`

nearest-neighbour implementation, restart after an unsuccessful construction attempt, and require the final hull to contain every input point.

We used three real geographic datasets: contiguous U.S. cities ($n = 219$), gray-wolf sightings ($n = 1054$), and coast-redwood sightings ($n = 1190$). We also used points sampled uniformly from a disk, as in the original paper (Moreira and Santos, 2007), and an eight-pointed star designed to create difficult concavities and force dead ends. Each synthetic family was tested at $n = 1,000, 5,000, 10,000$, and $50,000$ over ten random seeds; each real dataset received ten timing trials.

Every trial evaluated all four configurations on the same input, and their execution order was varied across trials to reduce timing bias. An untied warm-up preceded each timing series to exclude one-time library-loading costs, and intersection counts were collected separately so that counting did not affect the runtime measurements. For each run, we measured runtime, the number and values of k tested, the final k , the number of exact intersection tests, and whether the terminal fallback was used. We also computed the hull area ratio ρ and verified that the final hull contained every input point. Within each growth schedule, naive and bucketed checking produced identical ordered hulls in all 220 schedule-matched comparisons. No reported benchmark run used the fallback, so all timing and shape comparisons concern concave-hull outputs.

Figure 4 summarizes the paired results. Bucketing alone gives paired median speedups of 1.40–3.41 $\times$ and reduces exact-intersection predicates by 6.9–188.6 $\times$. Geometric growth gives 1.36–44.57 $\times$ speedups with naive checking and 1.21–43.52 $\times$ with bucketing. It reduces the number of attempts by 1.5–16.3 $\times$ and the sum of tested k values by 1.33–17.80 $\times$. Combining both changes gives 1.89–147.94 $\times$ speedups over the original baseline.

Geometric growth is not quality-preserving in the same sense as bucketing: it skips values of k . It returned the same hull as linear growth in only 11 of 110 paired inputs. In this experiment the other 99 hulls had larger ρ , and the median dataset-level increase ranged from 0.003 to 0.202. For coast redwood, for example, the median attempt count falls from 114 to 7, while final k rises from 116 to 192 and ρ rises from 0.359 to 0.422. These observations establish a practical runtime–shape tradeoff, not a monotone relationship between k and concavity.

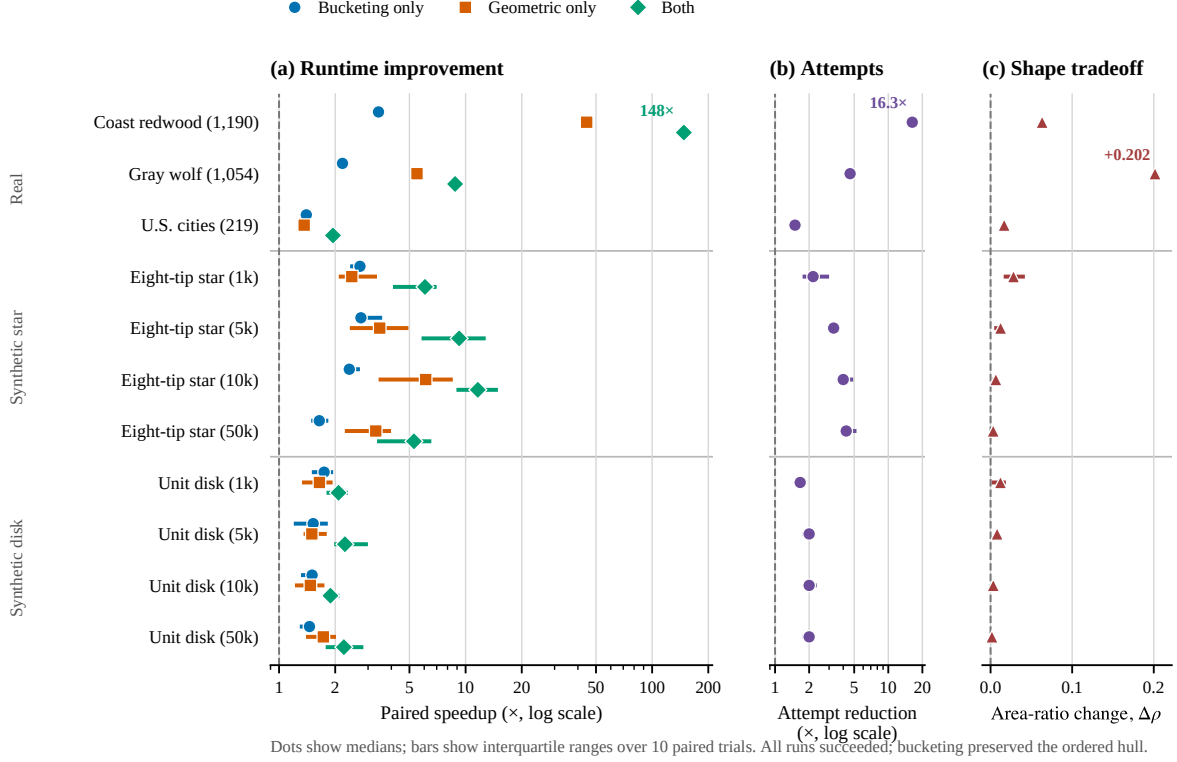


Figure 4: Results of the 2×2 factorial experiment. Speedups are relative to the original linear-growth, naive-intersection baseline. Dots show medians and bars show interquartile ranges over ten paired trials. Attempt reduction compares linear and geometric growth; attempts include the final successful construction, so attempts equal restarts plus one. The area-ratio change is $\Delta\rho = \rho_{\text{geometric}} - \rho_{\text{linear}}$; positive values indicate a larger enclosed area, not necessarily a higher-quality hull. All runs produced valid hulls, and bucketing preserved the ordered hull.

6 NON-MONOTONE EFFECTS OF K

The parameter k is usually described as a smoothness control: a larger value broadens the candidate set and tends to produce a boundary closer to the convex hull. There are two distinct monotonicity questions. First, does increasing k always make a valid hull less concave? Second, if a fixed- k attempt is valid, must every larger value also be valid? We give counterexamples to both claims.

6.1 Hull Shape Is Not Monotone

For a valid hull H of a point set S , define the area ratio

$$\rho(H) = \frac{\text{area}(H)}{\text{area}(\text{conv}(S))}, \quad 0 < \rho(H) \leq 1.$$

Smaller values mean that the boundary removes more empty area from the convex hull. For the eight-point set in Table 1, the convex-hull area is 0.32411. The

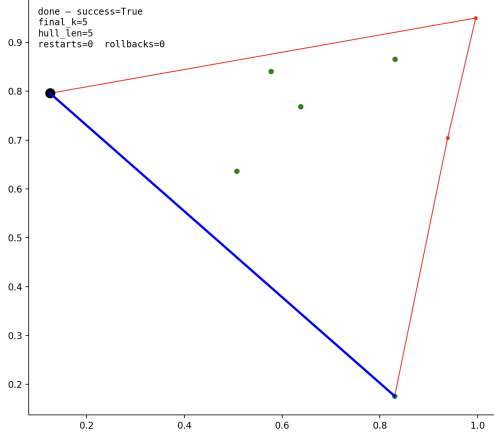
algorithm gives

$$\begin{aligned} \rho(H_5) &= \frac{0.32223}{0.32411} \approx 0.9942, \\ \rho(H_6) &= \frac{0.26187}{0.32411} \approx 0.8079. \end{aligned}$$

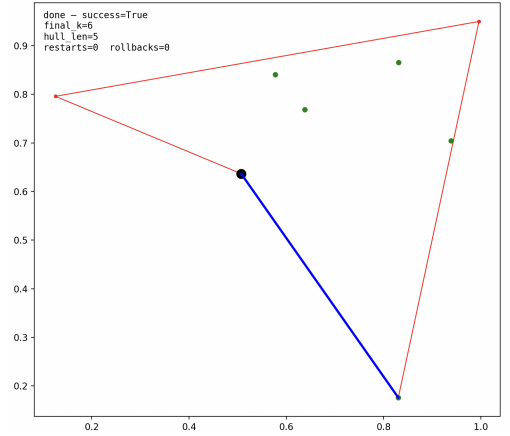
Both hulls enclose all eight input points, but increasing k from 5 to 6 produces the more concave hull shown in Figure 5. With $k = 6$, the algorithm can consider an additional neighbor and therefore skips a nearby point early in the construction. That point remains available and is selected later, pulling the boundary inward and creating a deeper concavity. Increasing k can therefore change the order in which points are selected rather than simply making the same hull smoother.

6.2 Fixed- k Validity Is Not Monotone

Let $V(k)$ be true when a fixed- k attempt successfully constructs a hull that encloses all input points. One possible way to find the smallest valid k would be binary search. However, binary search would require a



(a) $k = 5$: $\rho \approx 0.9942$.



(b) $k = 6$: $\rho \approx 0.8079$.

Figure 5: Increasing k can produce a more concave valid hull.

Table 1: Eight-point shape counterexample.

x	y	x	y
0.99603	0.94980	0.93863	0.70464
0.12534	0.79581	0.63777	0.76837
0.83024	0.17598	0.83103	0.86563
0.57695	0.84059	0.50715	0.63669

single validity threshold: values below the threshold would fail, while the threshold and every larger value would succeed.

The following six-point integer set shows that this requirement does not hold:

$$S_v = \{A = (9, 9), B = (7, 9), C = (0, 2), \\ D = (0, 8), E = (4, 7), F = (3, 1)\}.$$

The set has no three collinear points, and no point has two neighbours at the same distance. The result therefore does not depend on geometric degeneracy or nearest-neighbour tie-breaking. Direct execution gives

$$V(3) = \text{true}, \quad V(4) = \text{false}, \quad V(5) = \text{true}.$$

At $k = 3$, the walk $F \rightarrow E \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow F$ closes and contains all six points. At $k = 4$, the walk begins $F \rightarrow B \rightarrow A$, but every possible continuation from A to C , D , or E intersects the existing edge FB . The construction therefore reaches a dead end. At $k = 5 = n - 1$, the walk $F \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow F$ closes and encloses E . Figure 6 shows these three outcomes.

These outcomes show that fixed- k validity is not monotone. Binary search is therefore not a correct method for finding the smallest valid k : after observing the failure at $k = 4$, it could discard smaller values even though $k = 3$ succeeds. The geometric schedule does not require validity to be monotone. It simply increases k after each failed attempt until it finds

a valid hull or tests $k = n - 1$, where all remaining points are considered. Geometric growth may skip a smaller successful value; if the final all-candidates attempt fails, the algorithm uses the convex-hull fallback rather than repeating that value.

7 FUTURE WORK

Reducing discarded work. The restart analysis shows that an unsuccessful attempt discards the entire partial hull and repeats the construction with a larger value of k . An alternative would be to save the partial hull at selected points and, after a failure, return to one of these points and rebuild only the affected portion of the boundary. We refer to this partial-restart approach as *checkpoint trimming*. A complete treatment would need to determine where checkpoints should be placed, how to recover when a closed hull excludes points, and how to preserve containment and termination.

Amortized analysis of the full algorithm. The current analysis bounds each construction attempt separately and then sums those bounds across the tested values of k . An amortized analysis could instead track the total work spent constructing, testing, discarding, and reconstructing hull edges over the complete execution. This could provide a tighter description of the original restart algorithm and determine whether checkpoint trimming improves the asymptotic runtime or primarily reduces practical wasted work.

Candidate-selection criteria. A complementary direction is to reduce the number of failures that trig-

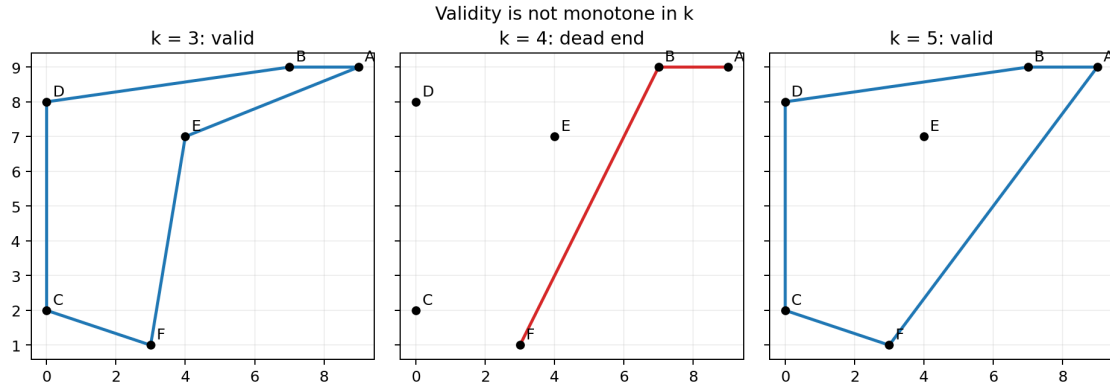


Figure 6: A six-point validity counterexample. The fixed- k construction is valid at $k = 3$, fails at $k = 4$, and is valid again at $k = 5$.

ger reconstruction. The current algorithm selects candidates using proximity and turning angle. Alternative criteria could also consider distance, local point density, intersection risk, or the position of the remaining points. These criteria could be evaluated by measuring their effects on failed attempts, discarded work, total runtime, and the shape of the resulting hull.

8 CONCLUSION

We revisited the Moreira–Santos k -nearest neighbours concave hull algorithm to explain its cost and improve its practical performance. One fixed- k attempt has an $O(kn^2)$ worst-case upper bound, while the original linear restart rule gives an $O(n^4)$ upper bound. Constant-ratio geometric growth changes the restart sum and lowers the corresponding upper bound to $O(n^3)$, although it may skip smaller successful values of k . Testing $k = n - 1$ once and then using a convex-hull fallback closes the original termination gap without changing either bound.

Uniform-grid spatial bucketing reduces exact intersection predicates without changing the resulting hull; across the tested real and synthetic datasets it produced paired median speedups of 1.40 – $3.41\times$ over naive checking. Geometric growth reduced attempts by 1.5 – $16.3\times$ and produced 1.36 – $44.57\times$ speedups, while usually selecting a different valid hull. The combined configuration was 1.89 – $147.94\times$ faster than the original linear-growth, naive-intersection baseline. Finally, k is not a monotone control variable. A larger value can produce a more concave valid hull, and fixed- k validity can change from true to false and back to true. The latter result rules out ordinary binary search for the smallest valid k . Together with the terminal fallback, forward geometric growth provides a finite search schedule, though it may skip smaller successful values of k .

REFERENCES

- Andrew, A. M. (1979). Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219.
- Duckham, M., Kulik, L., Worboys, M., and Galton, A. (2008). Efficient generation of simple polygons for characterizing the shape of a set of points in the plane. *Pattern Recognition*, 41(10):3224–3236.
- Edelsbrunner, H., Kirkpatrick, D., and Seidel, R. (1983). On the shape of a set of points in the plane. *IEEE Transactions on Information Theory*, 29(4):551–559.
- Franklin, W. R., Narayanaswami, C., Kankanhalli, M., Sun, D., Zhou, M.-C., and Wu, P. Y. F. (1989). Uniform grids: A technique for intersection detection on serial and parallel machines. In *Proceedings of Auto-Carto 9: Ninth International Symposium on Computer-Assisted Cartography*, pages 100–109, Baltimore, Maryland.
- Galton, A. and Duckham, M. (2006). What is the region occupied by a set of points? In *GIScience*, pages 81–98.
- Jarvis, R. A. (1973). On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21.
- Liao, Z., Liu, J., Shi, G., and Meng, J. (2021). Grid partition variable step alpha shapes algorithm. *Mathematical Problems in Engineering*.
- Moreira, A. and Santos, M. Y. (2007). Concave hull: A k -nearest neighbours approach for the computation of the region occupied by a set of points. In *Proceedings of the International Conference on Computer Graphics Theory and Applications (GRAPP)*, pages 61–68.